Extracción de 3 Características de una Imagen de Paisaje y Reconocer dichas Características en otras

Armando García
Introducción al Reconocimiento de Patrones
Instituto Tecnológico de Costa Rica
San José, Costa Rica
armandgp07@estudiantec.cr

Binjie Liang
Introducción al Reconocimiento de Patrones
Instituto Tecnológico de Costa Rica
San José, Costa Rica
bliang@estudiantec.cr

Alejandro Madrigal
Introducción al Reconocimiento de Patrones
Instituto Tecnológico de Costa Rica
San José, Costa Rica
alemarra99@estudiantec.cr

Esteban Secaida
Introducción al Reconocimiento de Patrones
Instituto Tecnológico de Costa Rica
San José, Costa Rica
est@estudiantec.cr

***Abstract*—Este trabajo presenta un enfoque básico para el reconocimiento de imágenes de paisajes mediante la extracción de características visuales simples. Utilizando herramientas como OpenCV y NumPy en Python, se propone un modelo basado en tres métricas principales: la proporción de cielo en el tercio superior de la imagen, la proporción de no-cielo en el tercio medio y la proporción de suelo o vegetación en el tercio inferior.**

El sistema se estructura en dos fases: entrenamiento y clasificación. Durante el entrenamiento, se calculan estadísticas como el promedio y la desviación estándar de los vectores de características extraídos de imágenes de paisajes. Posteriormente, en la fase de clasificación, nuevas imágenes son evaluadas mediante un sistema de puntaje que determina su pertenencia a la categoría de paisaje.

Los resultados obtenidos muestran que el modelo logra identificar correctamente patrones visuales en la mayoría de los casos, evidenciando la utilidad de técnicas simples de segmentación por color en el espacio HSV. No obstante, también se identifican limitaciones en escenarios donde las características visuales no siguen el patrón esperado, lo que sugiere la necesidad de incorporar métodos más robustos en trabajos futuros.

***Index Terms*—Scientific writing, Typesetting, Document creation, Syntax**

I. Introducción

Durante los últimos años el reconocimiento de imágenes ha estado ganando terreno en todas las áreas de la industria, desde actividades cotidianas que nosotros realizamos como tomar una simple fotografía hasta a áreas más complejas como la medicina o la inteligencia artificial.

En el presente documento, se presentará una síntesis de bajo nivel, algo básico, en cómo funcionan los reconocedores de imágenes. Para ello debemos de contar con herramientas para dicho trabajo; por ello, para este laboratorio, usaremos herramientas como el lenguaje de programación Python, junto con bibliotecas de procesamiento de imágenes, como lo es OpenCV [1] y computación científica de datos de arreglos como lo es Numpy [2].

Para de este trabajo, se requiere saber cómo funcionan internamente las herramientas y utilidades con las que vamos a estar trabajando, dado que si en algún día se deja de dar mantenimiento a dichas herramientas, podamos desarrollarlos por nuestra cuenta propia.

A. Propiedades básicas en una imagen

Una imagen digital puede entenderse como una representación bidimensional de datos, compuesta por una matriz de valores numéricos conocidos como píxeles. Cada píxel contiene información que describe características visuales como color, intensidad o brillo. Las imágenes digitales son funciones discretas definidas sobre un dominio finito, lo que permite su manipulación mediante algoritmos computacionales [3].

a) *Resolución espacial:*

La resolución espacial se refiere a la cantidad de píxeles que componen una imagen, normalmente expresada como ancho × alto (por ejemplo, 1920×1080). A mayor resolución, mayor nivel de detalle se puede representar. Sin embargo, esto también implica un mayor costo computacional en procesamiento y almacenamiento.

La resolución es un factor clave en aplicaciones como visión por computadora, ya que afecta directamente la precisión de los algoritmos de reconocimiento. [4]

b) *Intensidad y niveles de grises:*

En imágenes en escala de grises, cada píxel contiene un valor que representa la intensidad luminosa. Este valor generalmente se encuentra en un rango de 0 a 255, donde 0 representa negro y 255 blanco. Este rango se basa en imágenes de 8 bits por píxel.

La intensidad es fundamental para técnicas como la segmentación y detección de bordes, ya que permite identificar diferencias entre regiones de la imagen. [5]

c) *Modelos de color:*

Las imágenes a color utilizan modelos como RGB (Rojo, Verde, Azul), donde cada píxel está compuesto por tres valores que representan la intensidad de cada canal de color

[6]. Herramientas como OpenCV y NumPy manejan estas representaciones como arreglos multidimensionales.

Otros modelos como HSV (Hue, Saturation, Value) son utilizados en procesamiento de imágenes debido a su capacidad para separar la información de color de la iluminación. [6]

d) *Profundidad de bits:*

La profundidad de bits define la cantidad de información que puede almacenar cada píxel. Por ejemplo, una imagen de 8 bits puede representar 256 niveles de intensidad, mientras que una de 24 bits (8 bits por canal en RGB) puede representar millones de colores. [6]

Una mayor profundidad de bits permite una representación más precisa de la imagen, lo cual es especialmente importante en aplicaciones médicas o científicas.

e) *Ruido en imágenes:*

El ruido es una alteración aleatoria de los valores de los píxeles que puede ocurrir durante la captura, transmisión o procesamiento de la imagen. Existen diferentes tipos de ruido, como el ruido gaussiano o el ruido sal y pimienta. [6]

Según estudios en procesamiento de imágenes, la reducción de ruido es un paso fundamental previo al reconocimiento, ya que mejora la calidad de los datos de entrada.

II. Herramientas a Utilizar

Para poder extraer esas propiedades o características en las imágenes se debe contar con herramientas especializadas en ese trabajo.

Es por eso que, en este apartado haremos mención de utilidades que nos proporciona las herramientas/bibliotecas que vamos a utilizar de Python, los cuales son de OpenCV y Numpy.

A. Utilidades de OpenCV

La biblioteca de OpenCV proporciona una amplia gama de utilidades para procesar imágenes, de las cuales vamos a tomar más en consideración aquellos que permitan manipular y leer la información de los píxeles como también manipular el tamaño y colores de la imagen.

Funciones básicas como `shape` (obtener alto, ancho y número de canales), `imread` (leer imágenes) son esenciales para este trabajo, pero no son suficientes.

a) *Resize:*

Una utilidad de gran importancia es la de `resize`. Con esta nos permite re dimensionar una imagen a una resolución deseada.

El proceso comienza estableciendo una relación entre la imagen original y la imagen redimensionada. Para cada píxel en la imagen de salida, se calcula su posición correspondiente en la imagen de entrada.

Sean:

- W, H el ancho y alto de la imagen original
- W', H' el ancho y alto de la imagen redimensionada

El mapeo se define como:

$$x = x' * \left(\frac{W}{W'}\right) \tag{1}$$

$$y = y' * \left(\frac{H}{H'}\right) \tag{2}$$

Donde:

- (x', y') representan las coordenadas en la imagen redimensionada
- (x, y) representan las coordenadas en la imagen original

Este método se conoce como **mapeo inverso** (de destino a origen), el cual garantiza que cada píxel de la imagen resultante tenga un valor asignado.

Después de esto, se tiene que identificar a los píxeles vecinos, para luego aplicar el método de interpolación, de esta forma se puede asignar un valor resultante al píxel. [7]

b) *cvtColor:*

Este método permite convertir una imagen de un espacio de color a otro, como de BGR a escala de grises, HSV. Por especificaciones de OpenCV, se utiliza el espacio de color BGR (Blue, Green, Red) y no RGB (Red, Green, Blue).

La documentación de OpenCV menciona que para esta conversión se recorre píxel por píxel y se le aplica una transformación matemática que dependiendo del tipo de conversión, se aplica una fórmula diferente.

Para este laboratorio se va a utilizar el espacio de HSV.

Para calcular el los valores de HSV, se tiene lo siguiente [8]

Sea un píxel representado en formato BGR:

$$(B, G, R) \tag{3}$$

Paso 1: Obtener valores extremos

Se determina el valor máximo y mínimo entre los tres canales:

$$C_{\{\max\}} = \max(B, G, R) \tag{4}$$

$$C_{\{\min\}} = \min(B, G, R) \tag{5}$$

Paso 2: Calcular delta

Delta representa la diferencia entre el valor máximo y el mínimo:

$$\Delta = C_{\{\max\}} - C_{\{\min\}} \tag{6}$$

Este valor indica qué tan diferente es el color.

Paso 3: Calcular Saturation (S)

La saturación se define como:

$$S = \begin{cases} 0 & \text{si } C_{\max} = 0 \\ \frac{\Delta}{C_{\max}} & \text{en otro caso} \end{cases} \tag{7}$$

La saturación indica qué tan puro o intenso es el color.

Paso 4: Calcular Hue (H)

El tono depende de cuál canal tiene el valor máximo:

$$H = \begin{cases} 0 & \text{si } \Delta = 0 \\ 60 * \left(\frac{G-B}{\Delta}\right) & \text{si } C_{\max} = R \\ 60 * \left(\frac{B-R}{\Delta} + 2\right) & \text{si } C_{\max} = G \\ 60 * \left(\frac{R-G}{\Delta} + 4\right) & \text{si } C_{\max} = B \end{cases} \tag{8}$$

El resultado de H se expresa en grados (0° a 360°) y representa el tipo de color.

Para propósitos de este laboratorio se va a estar utilizando, o más bien dicho, se va a transformar las imágenes de BGR a HSV.

Cabe resaltar que por ser OpenCV, la escala de `Hue` tiene valores entre [0,179], `Saturation` y `Value` escala entre [0,255].

c) *InRange:*

Obtenido el «color» a partir de la conversión a HSV, por medio del método InRange se utiliza para realizar segmentación de imágenes, es decir, separar partes de una imagen según un rango de valores. [9]

```
mask = cv2.inRange(src, lower, upper)
```

No hay mucha ciencia detrás de este método más que identificar cuáles píxeles están en el rango que se le proporcionó.

d) *bitwise_or:*

El método `bitwise_or()` se utiliza para realizar una operación lógica OR a nivel de bits entre dos imágenes o máscaras. [10]

Se usa comúnmente para combinar máscaras binarias, por ejemplo, cuando quieres detectar varios rangos de color al mismo tiempo. [10]

```
result = cv2.bitwise_or(src1, src2)
```

B. Utilidades de Numpy

a) *count_nonzero:*

Esta utilidad cuenta cuántos elementos de un arreglo son distintos de cero, o más generalmente, cuántos elementos evalúan como verdaderos en un contexto booleano. [11]

Internamente, la función aplana el arreglo de entrada (a menos que se especifique un eje con el parámetro `axis`) y recorre sus elementos de forma vectorizada evaluando, para cada uno, si su valor es distinto de cero. El conteo se acumula en C y se devuelve como un entero o como un arreglo si se opera sobre un eje específico. Dado que la operación se realiza completamente en memoria contigua a nivel de C, el costo computacional es $O(n)$ donde n es el número total de elementos. [11]

En el contexto de este laboratorio, `count_nonzero` se aplica sobre máscaras binarias producidas por `cv2.inRange`. En esas máscaras, los píxeles que cumplen la condición de color tienen valor 255 (no cero) y los que no la cumplen tienen valor 0. Por lo tanto, contar los elementos no nulos equivale directamente a contar los píxeles que satisfacen la condición, lo que permite calcular proporciones como:

```
sky_ratio_top =
np.count_nonzero(sky_mask_top) /
sky_mask_top.size
```

donde `.size` devuelve el total de píxeles de la franja, obteniendo así la fracción de cielo presente en el tercio superior de la imagen.

b) *array:*

`np.array` es la función principal para construir la estructura de datos central de NumPy: el arreglo n-dimensional (`ndarray`). [12]

Internamente, cuando recibe una secuencia de Python (lista, tupla u otro objeto con interfaz de arreglo), NumPy infiere el tipo de dato (`dtype`) más apropiado aplicando reglas de promoción de tipos, y luego reserva un bloque de memoria **contiguo** donde almacena los valores en orden de filas (C order) por defecto. Esta disposición contigua es lo que permite que operaciones vectorizadas como sumas, comparaciones o multiplicaciones sobre todo el arreglo se ejecuten directamente en código compilado (C/BLAS), sin necesidad de bucles explícitos en Python, resultando en un rendimiento significativamente superior al de las listas nativas. [12]

El parámetro `copy=True` (valor por defecto) garantiza que el nuevo arreglo sea independiente del objeto original, evitando efectos secundarios por referencias compartidas. [12]

En este laboratorio, `np.array` cumple dos roles principales:

- **Definir rangos HSV**: los límites inferior y superior para la detección de cielo y vegetación se expresan como arreglos de tres elementos, lo que permite pasarlos directamente a `cv2.inRange`.

```
lower_sky = np.array([90, 30, 50])
upper_sky  = np.array([130, 255, 255])
```

- **Construir el vector de características**: las tres métricas calculadas por imagen se empaquetan en un único `ndarray` de forma (3,), que luego se apila en una matriz para calcular el promedio y la desviación estándar del modelo.

```
return np.array([sky_ratio_top,
non_sky_ratio_middle,
ground_ratio_bottom])
```

:

III. Metodología

El sistema se divide en dos etapas principales: entrenamiento y clasificación.

A. Adquisición y preparación de imágenes

Las imágenes de entrenamiento se almacenan en una carpeta ya definida, la cual debe contener únicamente fotografías de paisajes. Antes de extraer cualquier característica, cada imagen es sometida a dos operaciones de preprocesamiento:

1) **Lectura**: la imagen se carga desde disco mediante `cv2.imread`, que la representa internamente como un arreglo tridimensional en el espacio de color BGR.
2) **Redimensionado**: la imagen se escala a una resolución uniforme de 320 × 240 píxeles con `cv2.resize`. Esto garantiza que todas las imágenes aporten la misma cantidad de información y elimina la variabilidad introducida por distintos tamaños de captura.

B. Extracción de características

Cada imagen preprocesada se convierte al espacio de color HSV mediante `cv2.cvtColor`. Este espacio permite separar el tono del color (Hue) de la intensidad luminosa (Value), lo que facilita la detección robusta de colores específicos frente a cambios de iluminación.

La imagen HSV se divide en tres franjas horizontales de igual altura:

- **Tercio superior** (0 a $\frac{H}{3}$): región donde típicamente aparece el cielo.
- **Tercio medio** ($\frac{H}{3}$ a $2\frac{H}{3}$): zona de transición entre cielo y suelo.
- **Tercio inferior** ($2\frac{H}{3}$ a H): región donde se espera vegetación, suelo o edificaciones.

A partir de estas franjas se computan tres características escalares:

a) *Característica 1 — Proporción de cielo en el tercio superior:*

Se aplica `cv2.inRange` con el rango HSV $[90,, 30,, 50]$ – $[130,, 255,, 255]$, que corresponde a tonos azules propios del cielo despejado. La característica se calcula como:

$$f_1 = \frac{\text{píxeles dentro del rango azul en tercio superior}}{\text{total de píxeles de tercio superior}} \tag{9}$$

b) *Característica 2 — Proporción no-cielo en el tercio medio:*

Se aplica el mismo rango azul sobre el tercio medio y se toma el complemento:

$$f_2 = 1 - \frac{\text{píxeles azules en el tercio medio}}{\text{total de píxeles de tercio medio}} \tag{10}$$

Un valor alto de f_2 indica que la zona media no está dominada por el cielo, condición habitual en paisajes con horizonte visible.

c) *Característica 3 — Proporción de suelo o vegetación en el tercio inferior:*

Se combinan dos máscaras mediante `cv2.bitwise_or`:

- **Vegetación**: rango verde HSV $[35,, 40,, 40]$ – $[85,, 255,, 255]$.
- **Estructuras grises**: píxeles con saturación $S < 35$ y brillo $V > 80$, típicos de cemento y asfalto.

$$f_3 = \frac{\text{píxeles de vegetación} \cup \text{estructuras grises en tercio inferior}}{\text{total de píxeles de tercio inferior}} \tag{11}$$

El vector de características de cada imagen queda definido entonces como:

$$\boldsymbol{x} = [f_1, f_2, f_3] \tag{12}$$

IV. Resultados

El modelo se construyo a partir de imágenes de entrenamiento en las que se extrajeron tres características por cada foto: proporción de cielo azul en el tercio superior, proporción de no-cielo en el tercio medio y proporción de suelo (vegetación o estructuras grises) en el tercio inferior. Con esos vectores se calcularon el promedio y la desviación estándar, y luego cada imagen de prueba se clasifico con un sistema de puntaje (umbral de decisión: 3 puntos sobre 5 para etiquetar como **PAISAJE**).

En esta sección se presenta el análisis foto por foto en el orden solicitado. Dado que las imágenes de prueba no tienen etiqueta real en el nombre de archivo, la valoración de si el resultado fue bueno o malo se interpreta de forma cualitativa, según la coherencia visual de las características detectadas.

A. Foto 1

Fig. 1. Imagen de prueba: images (2).jpeg

Resultado del modelo: **NO PAISAJE**.

Explicación: el vector obtenido fue `[0.0, 1.0, 0.0]`, lo que indica que en la franja superior no se detecto cielo azul y en la franja inferior tampoco se detecto suelo/vegetación/estructura. Este patrón es poco compatible con una escena de paisaje típica. Evaluación: **bueno** (la decisión del modelo es coherente con las características extraídas).

B. Foto 2

Fig. 2. Imagen de prueba: images.jpeg

Resultado del modelo: **PAISAJE**.

Explicación: el vector fue `[0.93953125, 0.44488281249999995, 0.5746484375]`. Se observa una presencia muy alta de cielo en la parte superior y una proporción importante de suelo en la parte inferior, dos rasgos esperables en imágenes de paisaje. Evaluación: **bueno** (el resultado es consistente con una estructura visual de paisaje).

C. Foto 3

Fig. 3. Imagen de prueba: images (1).jpeg

Resultado del modelo: **NO PAISAJE**.

Explicación: el vector fue `[0.129609375, 0.823046875, 0.08359375]`. Aunque hay algo de cielo, la proporción de suelo detectada en el tercio inferior es baja (menor a 0.10), lo cual penaliza la hipótesis de paisaje en la lógica del clasificador. Evaluación: **aceptable/bueno** (la predicción se alinea con el criterio implementado).

D. Foto 4

Fig. 4. Imagen de prueba: Imagen 1.jpg.jpeg

Resultado del modelo: **PAISAJE**.

Explicación: el vector obtenido fue `[0.979140625, 0.5254687499999999, 0.68859375]`. El valor de cielo en la zona superior es muy alto y la zona inferior presenta una fracción considerable de suelo/estructura, por lo que la imagen encaja bien con el patrón de paisaje aprendido. Evaluación: **bueno**.

E. Foto 5

Fig. 5. Imagen de prueba: 3.jpeg

Resultado del modelo: **PAISAJE**.

Explicación: el vector fue `[0.3376171875, 0.5685546875, 0.002578125]`. Aunque el sistema la clasifico como paisaje, la proporción de suelo detectada es prácticamente nula, lo que sugiere una posible inconsistencia con el comportamiento esperado para una escena de paisaje completa. Evaluación: **potencialmente malo** (posible falso positivo).

El análisis por imagen muestra decisiones en general coherentes en cuatro casos y un caso dudoso. Para confirmar formalmente si cada resultado fue correcto o incorrecto, se requiere incorporar etiquetas reales de prueba (por ejemplo con prefijos `paisaje_` y `nopaisaje_`) y re-calcular métricas cuantitativas.

V. Conclusiones

A lo largo de este trabajo se desarrolló un modelo sencillo de reconocimiento de imágenes enfocado en la identificación de paisajes, utilizando técnicas básicas de procesamiento digital de imágenes. Se demostró que, a partir de la segmentación por color en el espacio HSV y la división estructurada de la imagen en regiones horizontales, es posible extraer características representativas que permiten distinguir entre imágenes de paisaje y no paisaje.

El uso de herramientas como OpenCV y NumPy facilitó la implementación del sistema, permitiendo manipular imágenes y realizar cálculos de manera eficiente. Asimismo, la definición de un vector de características basado en proporciones de color en diferentes secciones de la imagen resultó ser una estrategia efectiva para capturar patrones visuales típicos de los paisajes.

Los resultados evidencian que el modelo presenta un desempeño mayormente coherente con la lógica planteada, logrando clasificaciones correctas en la mayoría de los casos analizados. Sin embargo, también se observaron limitaciones, como la presencia de falsos positivos, lo que pone en evidencia que el enfoque basado únicamente en reglas simples puede no ser suficiente para escenarios más complejos.

References

[1] OpenCV Team, «OpenCV». [En línea]. Disponible en: https://opencv.org/

[2] NumPy Developers, «NumPy». [En línea]. Disponible en: https://numpy.org/es/

[3] A. K. Jain, *Fundamentals of Digital Image Processing*. Prentice Hall, 1989.

[4] IEEE, «IEEE Xplore Digital Library - Image Processing». [En línea]. Disponible en: https://ieeexplore.ieee.org/

[5] M. Sonka, V. Hlavac, y R. Boyle, *Image Processing, Analysis, and Machine Vision*, 4.ª ed. Cengage Learning, 2014.

[6] R. C. Gonzalez y R. E. Woods, *Digital Image Processing*, 4.ª ed. Pearson, 2018.

[7] OpenCV Developers, «Geometric Image Transformations». [En línea]. Disponible en: https://docs.opencv.org/4.x/da/d54/group__imgproc__transform.html

[8] ITOHI, «HSV/HSL Color Spaces and Conversion Algorithm». [En línea]. Disponible en: https://itohi.com/snippets/colors/hsv-hsl/

[9] OpenCV Developers, «Thresholding Operations using inRange». [En línea]. Disponible en: https://docs.opencv.org/4.x/da/d97/tutorial_threshold_inRange.html

[10] OpenCV Developers, «Bitwise Operations». [En línea]. Disponible en: https://docs.opencv.org/4.x/d0/d86/tutorial_py_image_arithmetics.html

[11] Numpy Developers, «numpy.count_nonzero — NumPy v2.2 Manual». [En línea]. Disponible en: https://numpy-org.translate.goog/doc/2.2/reference/generated/numpy.count_nonzero.html?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc

[12] Numpy Developers, «Numpy.Array — NumPy v2.4 Manual». [En línea]. Disponible en: https://numpy-org.translate.goog/doc/stable/reference/generated/numpy.array.html?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc